Week5_Day_3_Handson_Csharp_Gurunath_Rageni

Level-1 Problem 1:

Scenario

A financial application needs to process multiple reports simultaneously to reduce waiting time. Instead of executing tasks sequentially, the system should run them concurrently using **C# Tasks** so that reports are generated faster.

Requirements

1. Create three methods:
    a. GenerateSalesReport()
    b. GenerateInventoryReport()
    c. GenerateCustomerReport()
2. Each method should simulate processing time using Thread.Sleep() or Task.Delay().
3. Execute all three operations **concurrently using Task**.
4. Display a message when each report starts and when it finishes.
5. Display a final message once all reports are completed.

Technical Constraints

- Use **Task class from System.Threading.Tasks**.
- Use **Task.Run()** to execute methods.
- Use Task.WaitAll() or await **Task.WhenAll()** to wait for completion.
- The program must run in a **Console Application**.

Expectations

The program should start multiple report-generation tasks simultaneously and display completion messages for each report along with a final message once all tasks are completed.

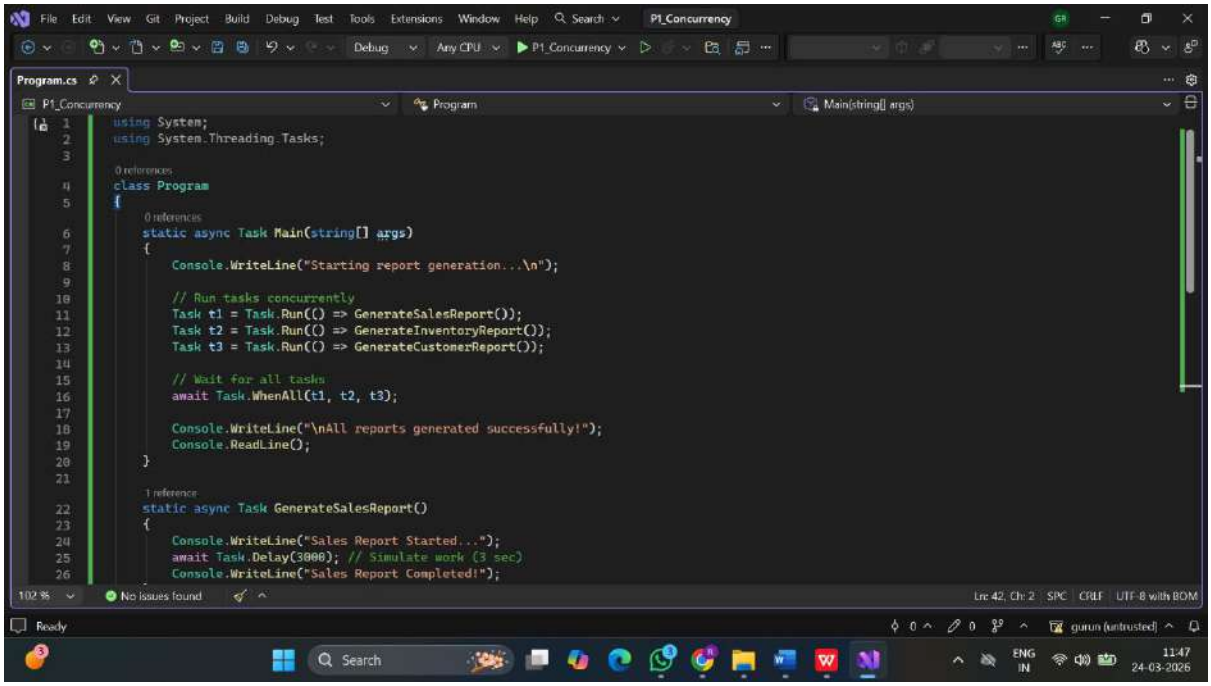
Learning Outcome

Students will learn:

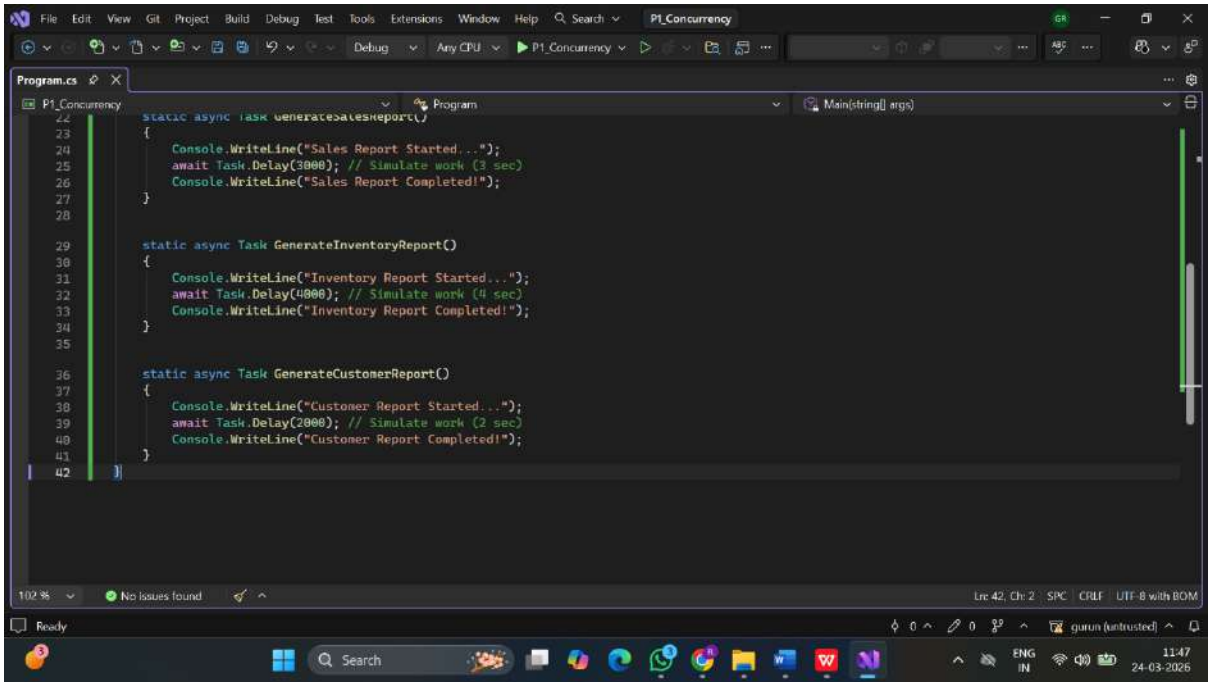
- How to create and run **Tasks in C#**
- How to execute **multiple operations concurrently**

How to **wait for multiple tasks to complete**

Code Screenshot:

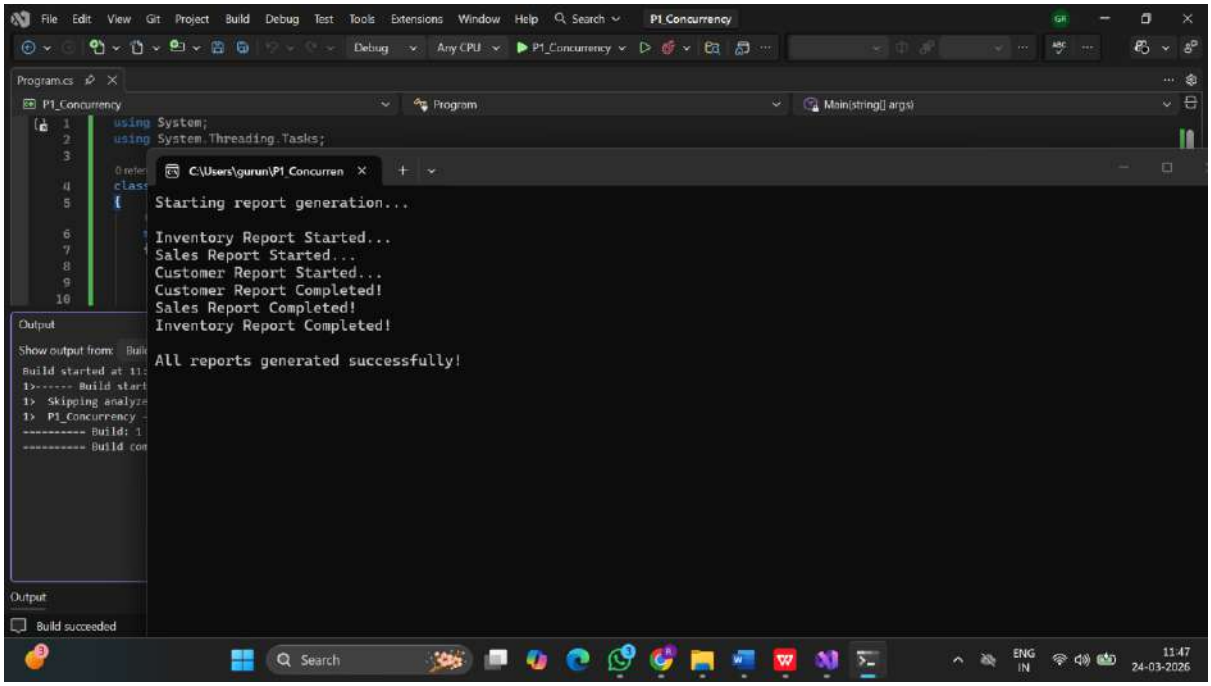


```
using System;
using System.Threading.Tasks;

0 references
class Program
{
    0 references
    static async Task Main(string[] args)
    {
        Console.WriteLine("Starting report generation...\n");

        // Run tasks concurrently
        Task t1 = Task.Run(() => GenerateSalesReport());
        Task t2 = Task.Run(() => GenerateInventoryReport());
        Task t3 = Task.Run(() => GenerateCustomerReport());

        // Wait for all tasks
        await Task.WhenAll(t1, t2, t3);

        Console.WriteLine("\nAll reports generated successfully!");
        Console.ReadLine();
    }

    1 reference
    static async Task GenerateSalesReport()
    {
        Console.WriteLine("Sales Report Started...");
        await Task.Delay(3000); // Simulate work (3 sec)
        Console.WriteLine("Sales Report Completed!");
```



```
    static async Task GenerateSalesReport()
    {
        Console.WriteLine("Sales Report Started...");
        await Task.Delay(3000); // Simulate work (3 sec)
        Console.WriteLine("Sales Report Completed!");
    }


    static async Task GenerateInventoryReport()
    {
        Console.WriteLine("Inventory Report Started...");
        await Task.Delay(4000); // Simulate work (4 sec)
        Console.WriteLine("Inventory Report Completed!");
    }


    static async Task GenerateCustomerReport()
    {
        Console.WriteLine("Customer Report Started...");
        await Task.Delay(2000); // Simulate work (2 sec)
        Console.WriteLine("Customer Report Completed!");
    }
}
```

Output:



Level-1 Problem 2: Asynchronous File Logger

Scenario:
An application writes logs to a file whenever an event occurs. Writing logs synchronously can slow down the application. Asynchronous file writing improves performance.

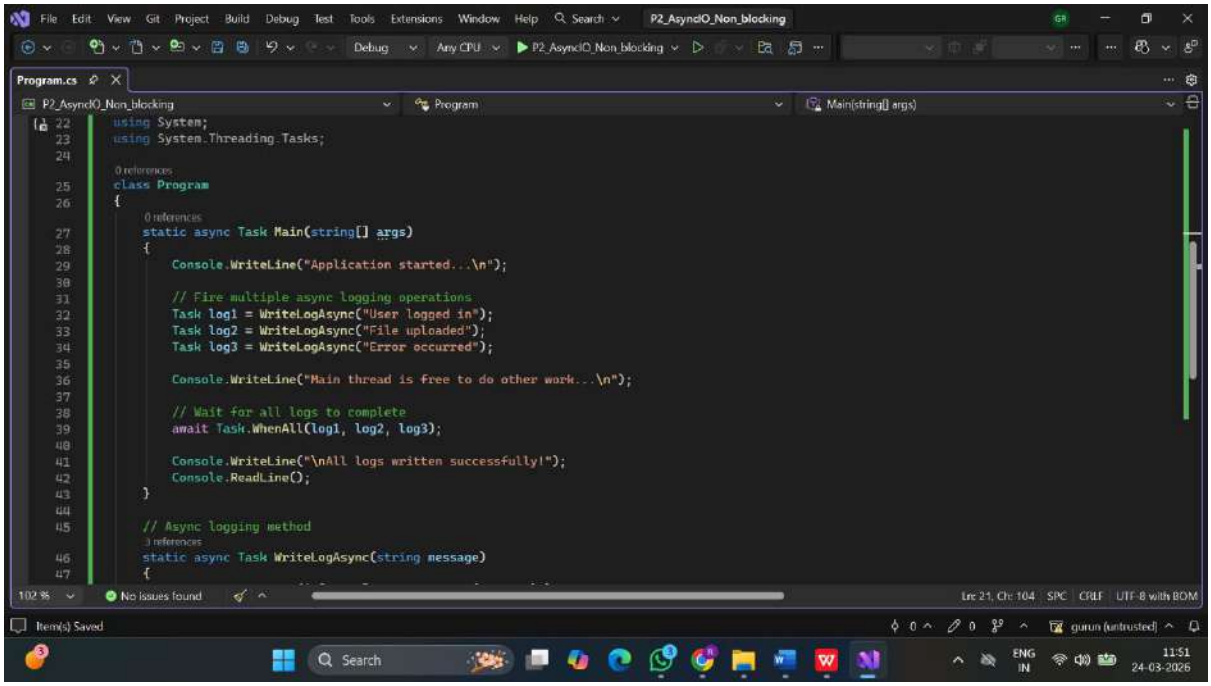
Requirements:
- Create an asynchronous method WriteLogAsync(string message).
- The method should simulate file writing using Task.Delay().
- Call this method multiple times to simulate logging different events.

Technical Constraints:
- Use async and await keywords.
- Use Task.Delay() to simulate file I/O.
- Use a console application.

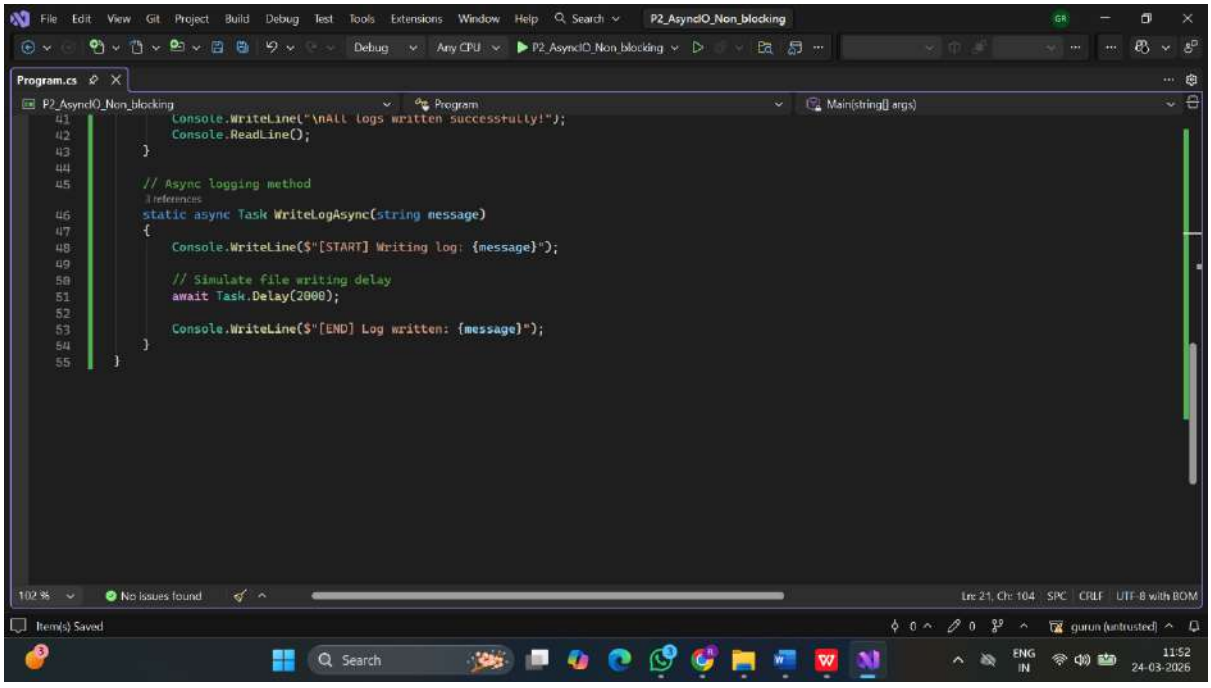
Expectations:
- Logs should be written asynchronously.
- The main thread should remain responsive while logging operations occur.

Learning Outcome:
Students will learn how asynchronous operations improve performance when dealing with I/O operations.

Code Screenshots:

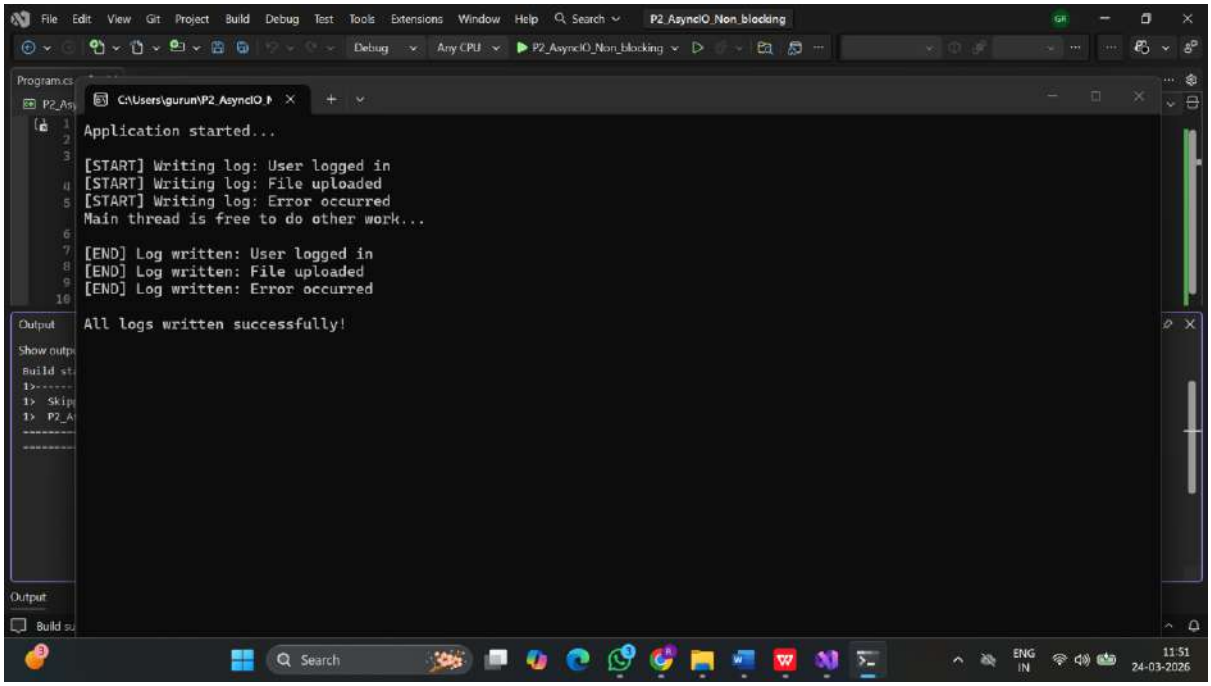


```
using System;
using System.Threading.Tasks;

class Program
{
    static async Task Main(string[] args)
    {
        Console.WriteLine("Application started...\n");

        // Fire multiple async logging operations
        Task log1 = WriteLogAsync("User logged in");
        Task log2 = WriteLogAsync("File uploaded");
        Task log3 = WriteLogAsync("Error occurred");

        Console.WriteLine("Main thread is free to do other work...\n");

        // Wait for all logs to complete
        await Task.WhenAll(log1, log2, log3);

        Console.WriteLine("\nAll logs written successfully!");
        Console.ReadLine();
    }

    // Async logging method
    static async Task WriteLogAsync(string message)
    {
```



```
        Console.WriteLine("\nAll logs written successfully!");
        Console.ReadLine();
    }

    // Async logging method
    static async Task WriteLogAsync(string message)
    {
        Console.WriteLine($"[START] Writing log: {message}");

        // Simulate file writing delay
        await Task.Delay(2000);

        Console.WriteLine($"[END] Log written: {message}");
    }
}
```

Output:



Level-2 Problem 1: Asynchronous Order Processing System

Scenario:
An e-commerce system processes customer orders. Each order requires multiple steps such as payment verification, inventory check, and order confirmation. These steps involve delays and should be handled asynchronously.

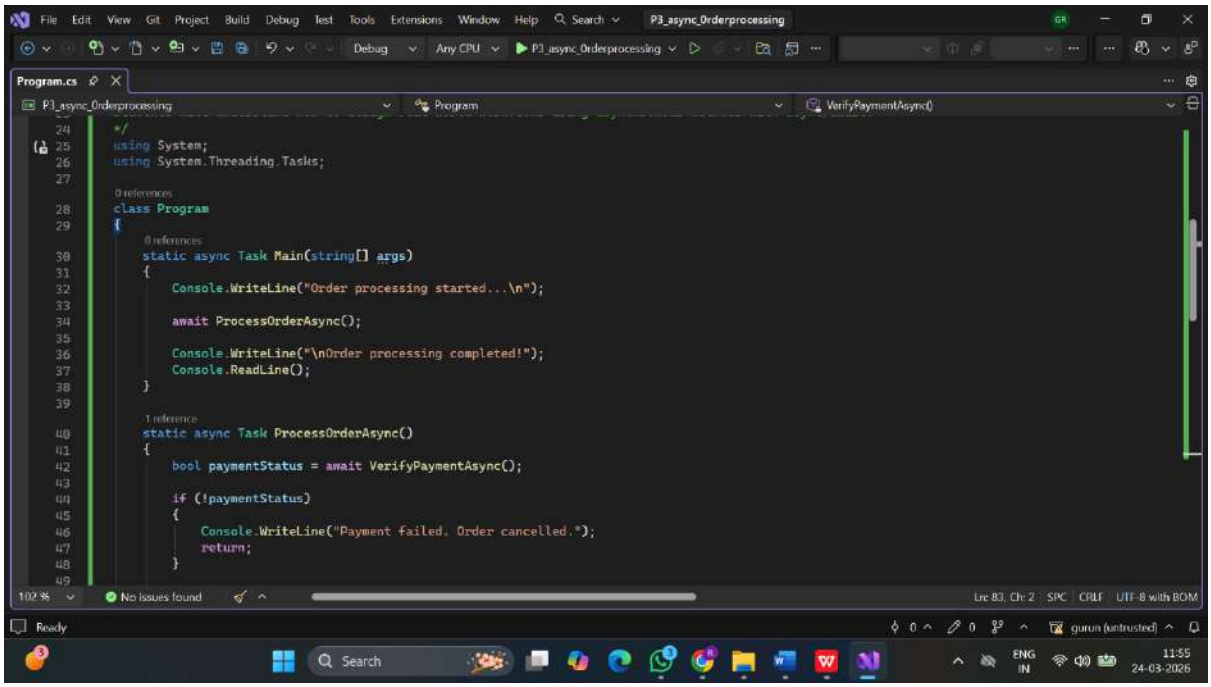
Requirements:
- Create asynchronous methods:
  - VerifyPaymentAsync()
  - CheckInventoryAsync()
  - ConfirmOrderAsync()
- Simulate processing delays using Task.Delay().
- Execute steps asynchronously while maintaining the logical order of operations.

Technical Constraints:
- Use async and await.
- Use Task-based asynchronous methods.
- Implement using a console application.

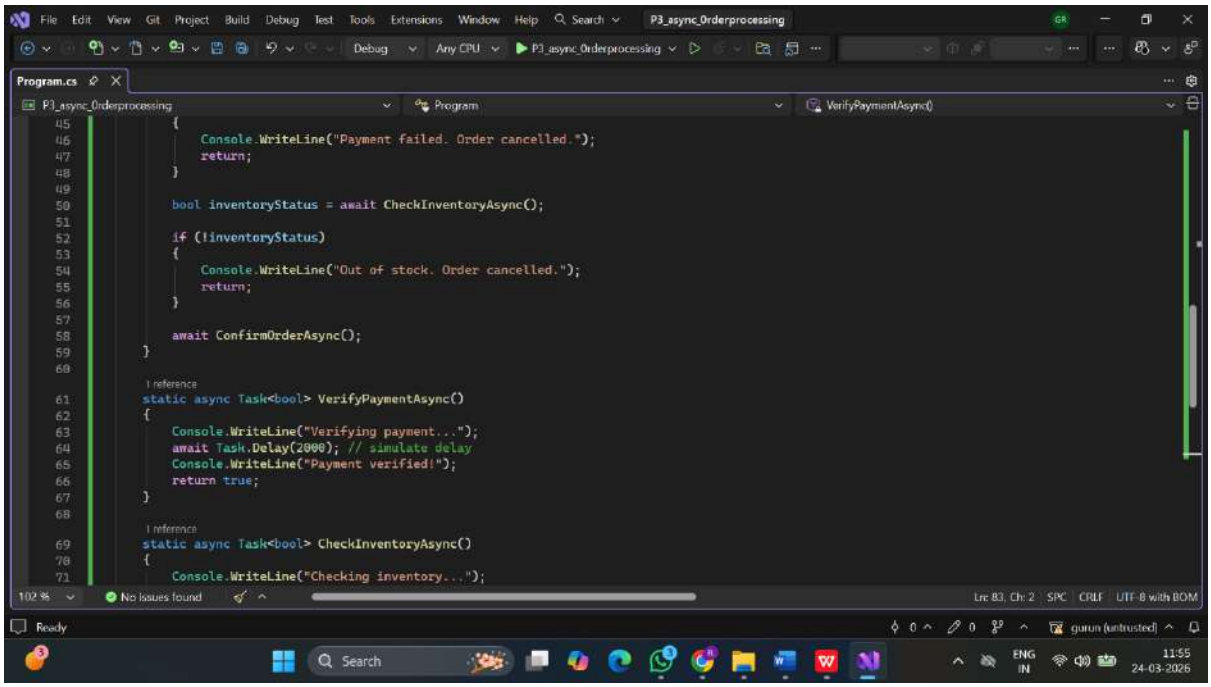
Expectations:
- Each processing step should run asynchronously.
- The program should display messages indicating the progress of order processing.

Learning Outcome:
Students will understand how to design real-world workflows using asynchronous methods with async/await.

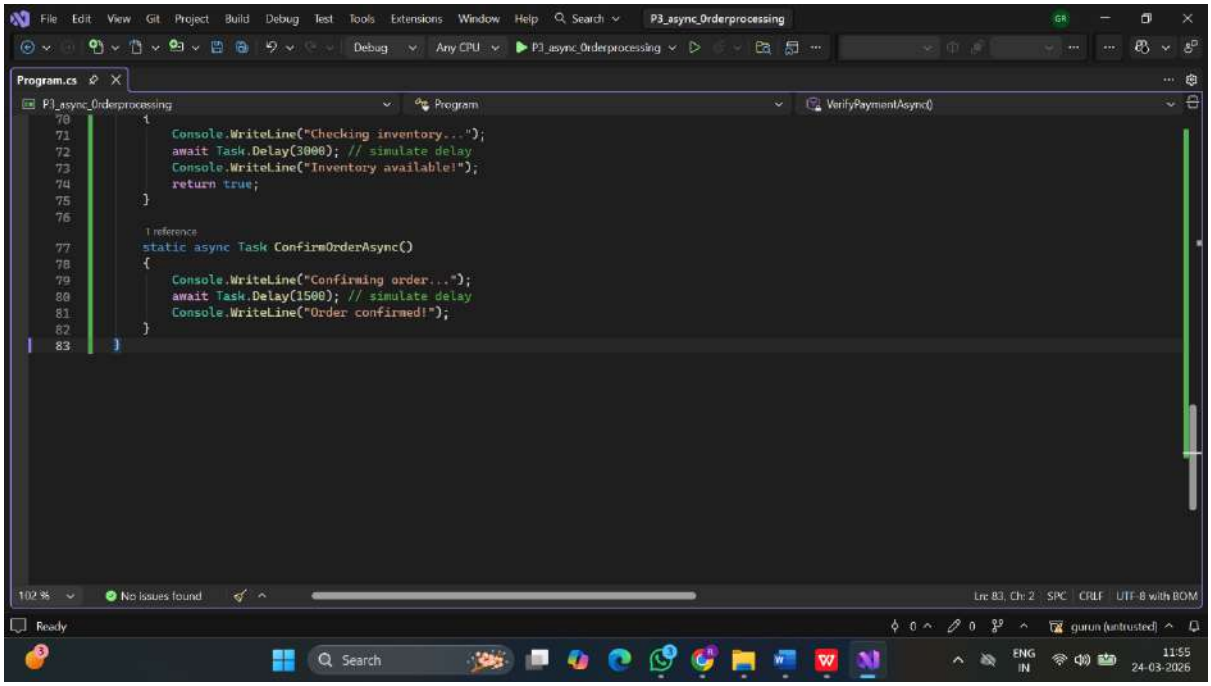
Code Screenshot:



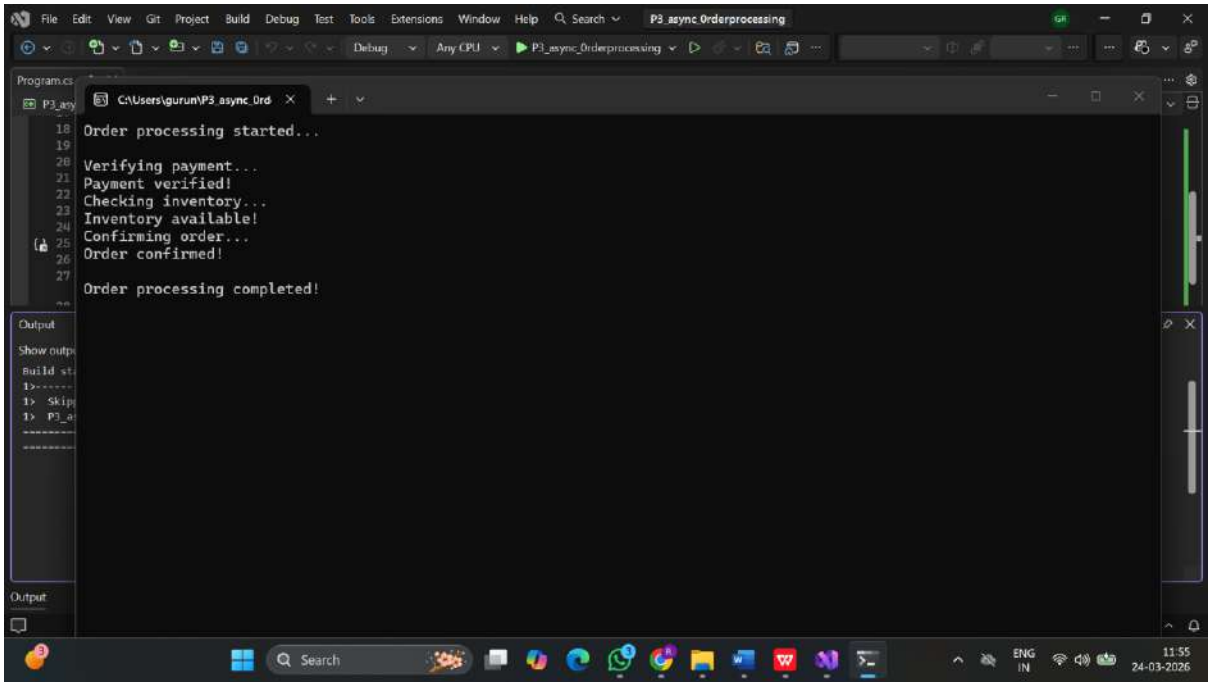
```
*/
using System;
using System.Threading.Tasks;

class Program
{
    static async Task Main(string[] args)
    {
        Console.WriteLine("Order processing started...\n");

        await ProcessOrderAsync();

        Console.WriteLine("\nOrder processing completed!");
        Console.ReadLine();
    }

    static async Task ProcessOrderAsync()
    {
        bool paymentStatus = await VerifyPaymentAsync();

        if (!paymentStatus)
        {
            Console.WriteLine("Payment failed. Order cancelled.");
            return;
        }
```



```
        {
            Console.WriteLine("Payment failed. Order cancelled.");
            return;
        }

        bool inventoryStatus = await CheckInventoryAsync();

        if (!inventoryStatus)
        {
            Console.WriteLine("Out of stock. Order cancelled.");
            return;
        }

        await ConfirmOrderAsync();
    }

    static async Task<bool> VerifyPaymentAsync()
    {
        Console.WriteLine("Verifying payment...");
        await Task.Delay(2000); // simulate delay
        Console.WriteLine("Payment verified!");
        return true;
    }

    static async Task<bool> CheckInventoryAsync()
    {
        Console.WriteLine("Checking inventory...");
```



Output:



Level-2 Problem 2: Debugging Incorrect Discount Calculation

Scenario

A retail application calculates the final price of products after applying a discount. Recently, users reported that the final price shown by the application is incorrect. The development team needs to **debug the application to identify where the incorrect calculation is happening**.

Requirements

- Create a console application that calculates the final product price.
- The program should accept:
    - Product Name
    - Product Price
    - Discount Percentage
- The final price should be calculated using the formula:

FinalPrice = Price – (Price × Discount / 100)

- Use **debugging tools** to verify that the calculation is correct.
- Place **breakpoints** and inspect variable values during execution.

Technical Constraints

- Use **Visual Studio Debugging Tools**.
- Use **breakpoints, step over, step into, and watch window**.
- Implement the solution using a **.NET console application**.

Expectations

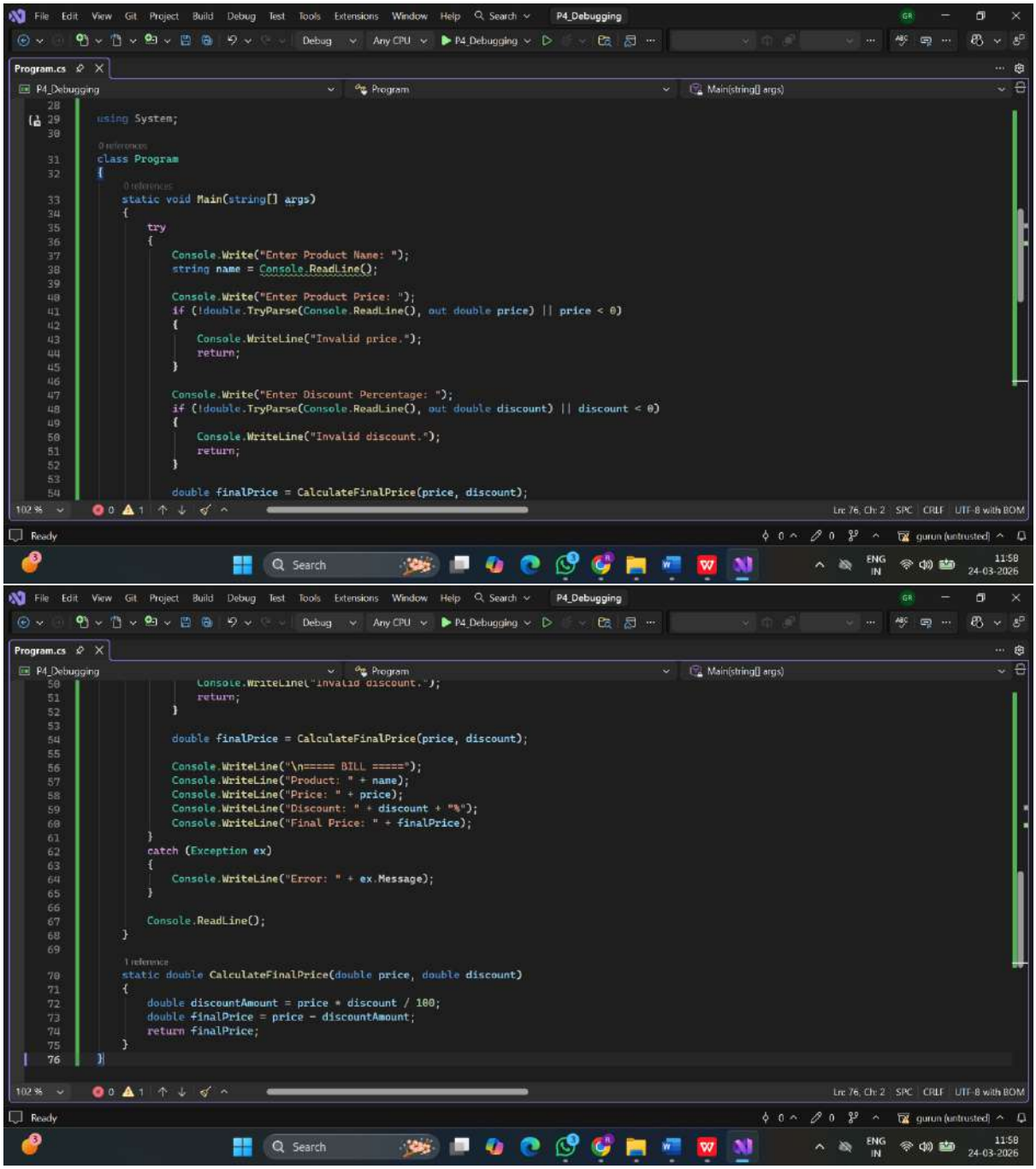
- Students should run the program in **debug mode**.
- They should track variable values and confirm that the discount calculation is correct.
- If incorrect results appear, students should identify the faulty logic.

Learning Outcome

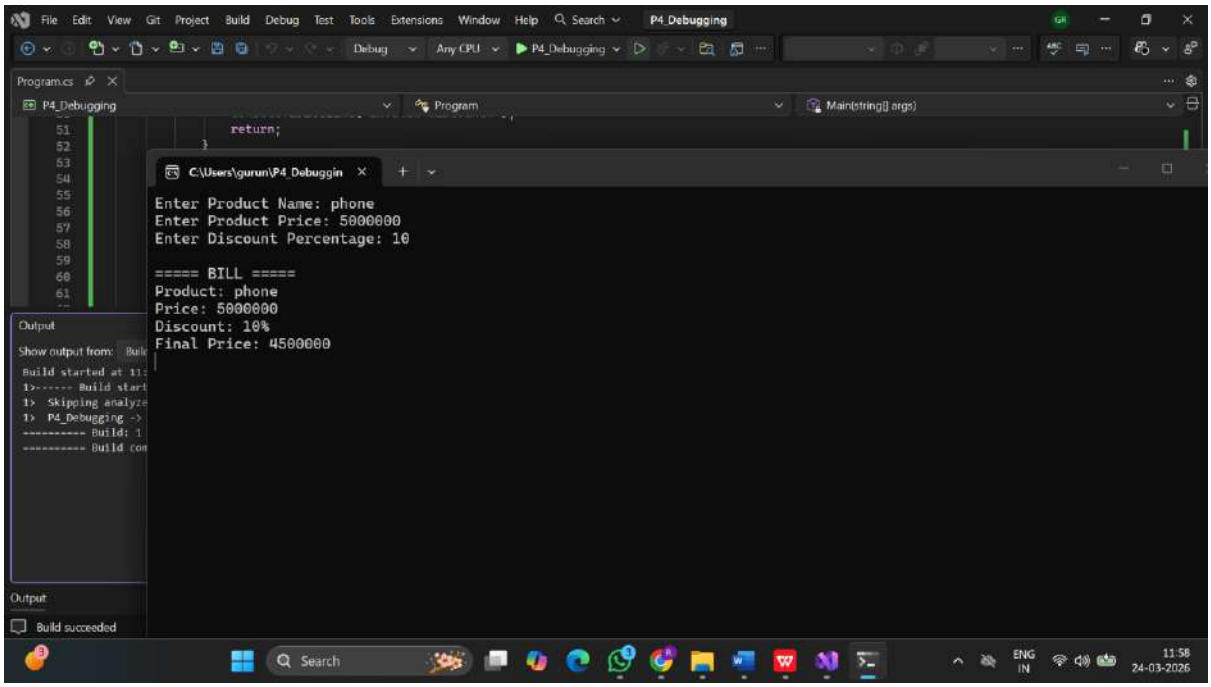
Students will learn how to:

- Use **breakpoints effectively**.
- Inspect **variable values during program execution**.
- Identify logical errors using **debugging techniques**.

Code Screenshot:



Output:



Level-2 Problem 3: Application Tracing for Order Processing

Scenario

An e-commerce application processes customer orders. Sometimes the order processing fails, but developers are unable to identify where the failure occurs. The team decides to implement **Tracing** to monitor the execution flow and diagnose the issue.

Requirements

- Create a console application that simulates **order processing**.
- The order processing should include the following steps:
    1. Validate Order
    2. Process Payment
    3. Update Inventory
    4. Generate Invoice
- Use **Trace class** to log messages at each step of the process.
- Display trace messages showing the execution flow.

Technical Constraints

- Use **System.Diagnostics.Trace**.
- Write trace messages using:
    - Trace.WriteLine()

- Trace.TraceInformation()

- Configure a **TextWriterTraceListener** to store trace logs in a file.
- Implement the solution using **.NET console application**.

Expectations

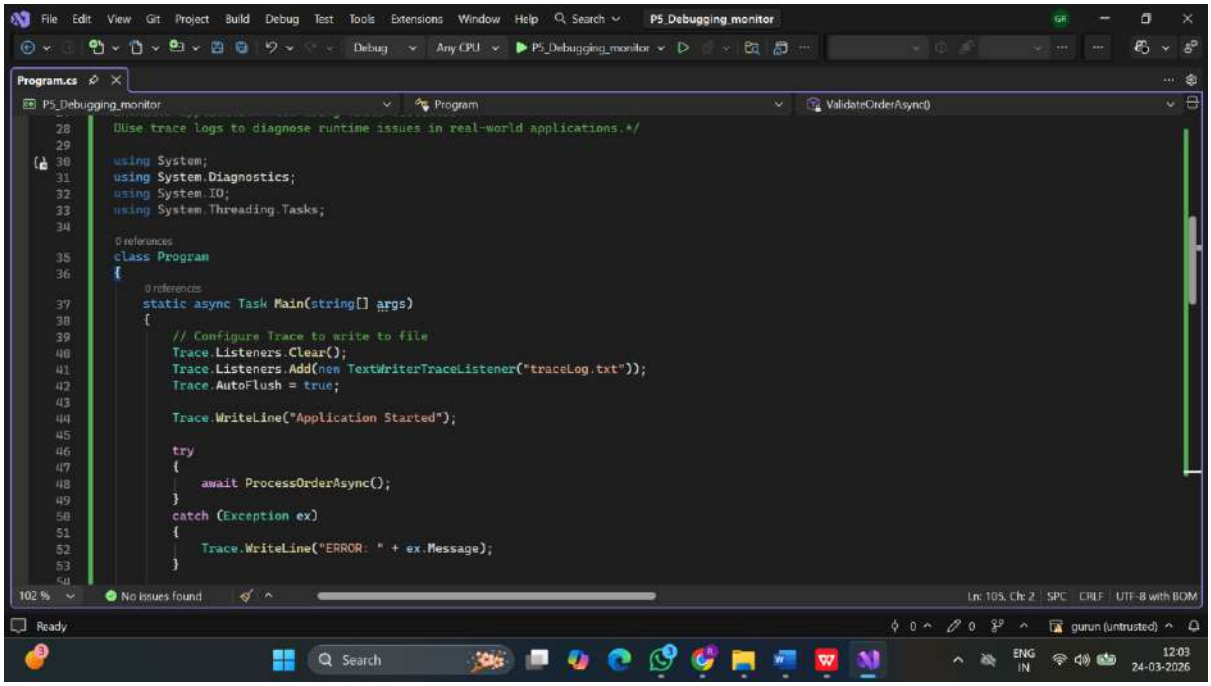
- The application should log messages for each stage of order processing.
- Trace logs should help developers identify where failures occur.
- The trace output should be stored in a **log file**.

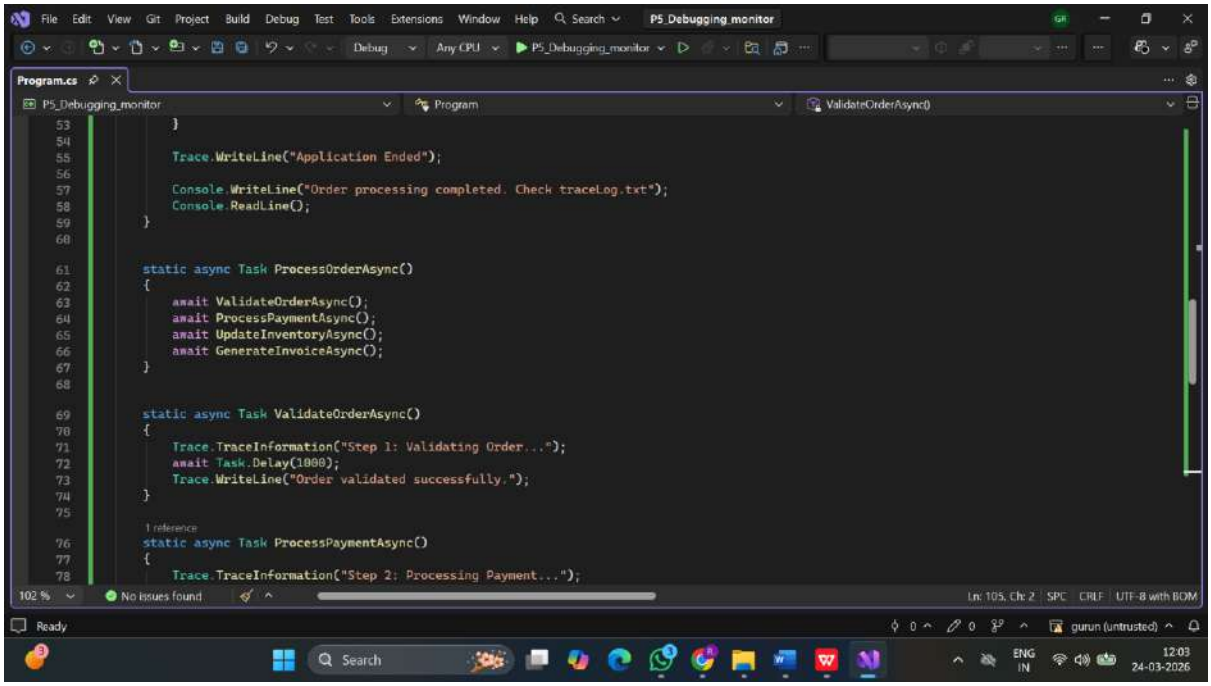
Learning Outcome

Students will learn how to:

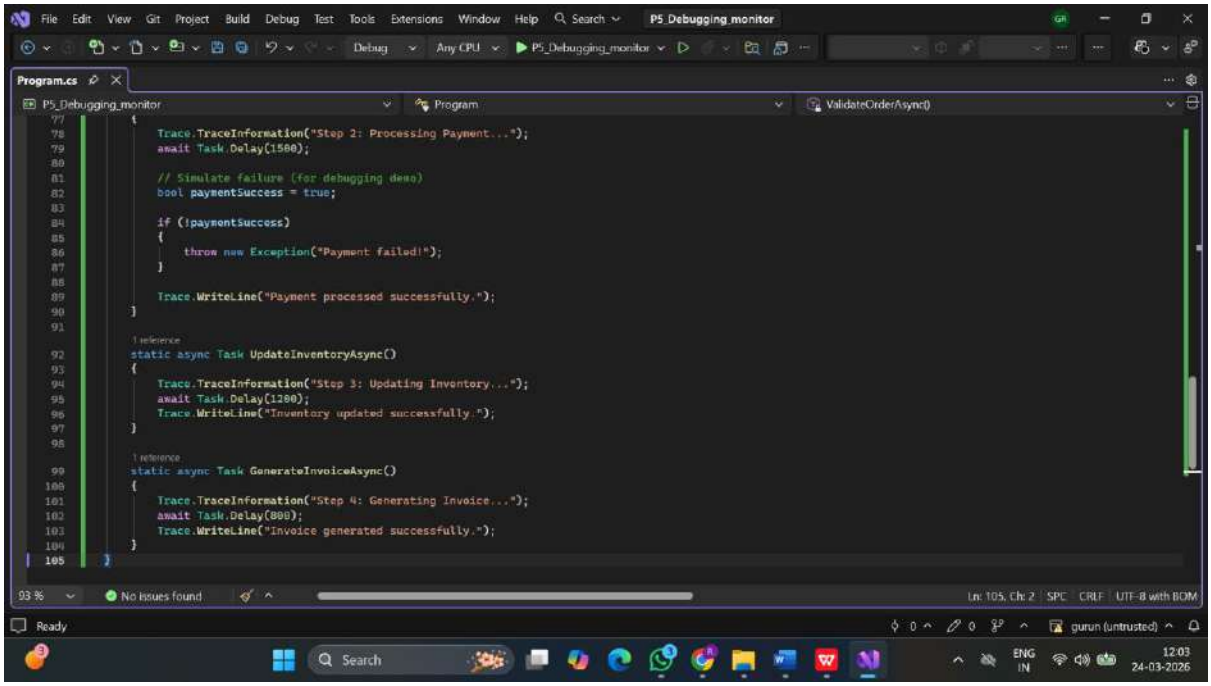
- Implement **application tracing** using System.Diagnostics.
- Monitor application flow using **Trace listeners**.
- Use trace logs to **diagnose runtime issues in real-world applications**.

Code Screenshot:



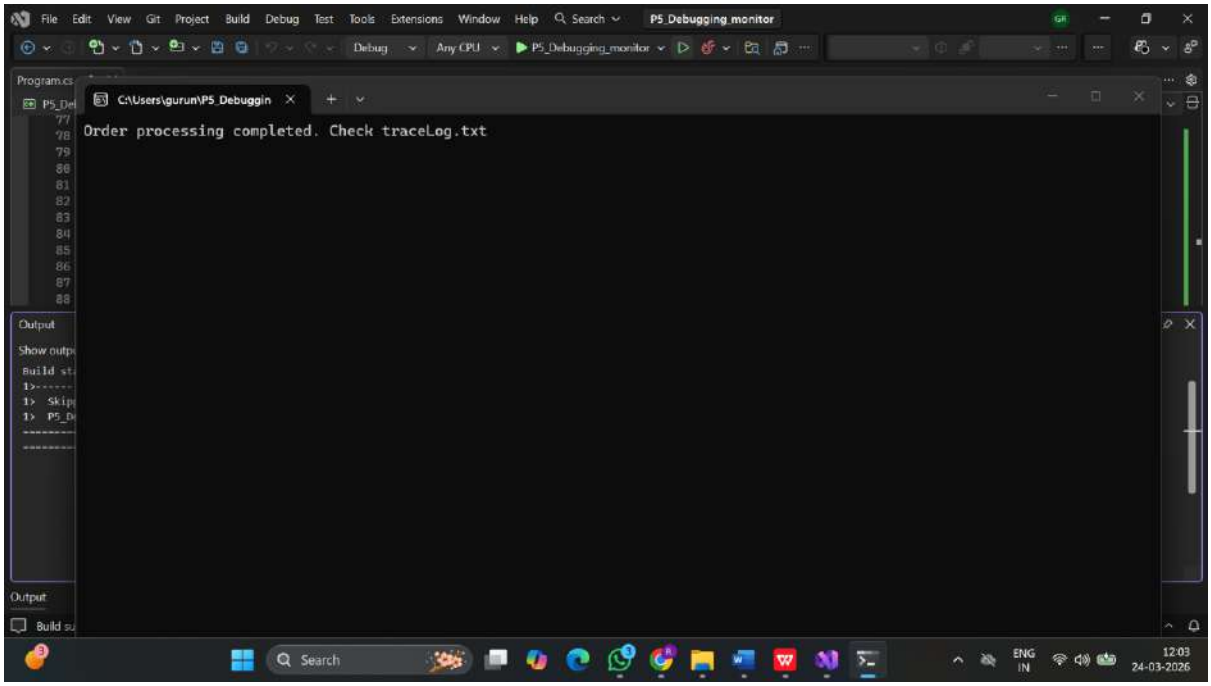


```
        }

        Trace.WriteLine("Application Ended");

        Console.WriteLine("Order processing completed. Check traceLog.txt");
        Console.ReadLine();
    }


    static async Task ProcessOrderAsync()
    {
        await ValidateOrderAsync();
        await ProcessPaymentAsync();
        await UpdateInventoryAsync();
        await GenerateInvoiceAsync();
    }


    static async Task ValidateOrderAsync()
    {
        Trace.TraceInformation("Step 1: Validating Order...");
        await Task.Delay(1000);
        Trace.WriteLine("Order validated successfully.");
    }

    1 reference
    static async Task ProcessPaymentAsync()
    {
        Trace.TraceInformation("Step 2: Processing Payment...");
```



```
    {
        Trace.TraceInformation("Step 2: Processing Payment...");
        await Task.Delay(1500);

        // Simulate failure (for debugging demo)
        bool paymentSuccess = true;

        if (!paymentSuccess)
        {
            throw new Exception("Payment failed!");
        }

        Trace.WriteLine("Payment processed successfully.");
    }

    1 reference
    static async Task UpdateInventoryAsync()
    {
        Trace.TraceInformation("Step 3: Updating Inventory...");
        await Task.Delay(1200);
        Trace.WriteLine("Inventory updated successfully.");
    }

    1 reference
    static async Task GenerateInvoiceAsync()
    {
        Trace.TraceInformation("Step 4: Generating Invoice...");
        await Task.Delay(800);
        Trace.WriteLine("Invoice generated successfully.");
    }
}
```

Output:


File Edit View Git Project Build Debug Test Tools Extensions Window Help Search
P5_Debugging_monitor
Debug
Any CPU
P5_Debugging_monitor
C:\Users\gurun\P5_Debuggin
Order processing completed. Check traceLog.txt
Output
Search
ENG
IN
12:03
24-03-2026